LICENCE PC-P · FILIÈRE E&S · SEMESTRE 6

TP1 — Analyse Spectrale & Synthèse de Fourier

Traitement du Signal · Prof. Said AGOUNAD · 2025-2026

Exercice 1 — Analyse de Fourier d'un signal *connu*

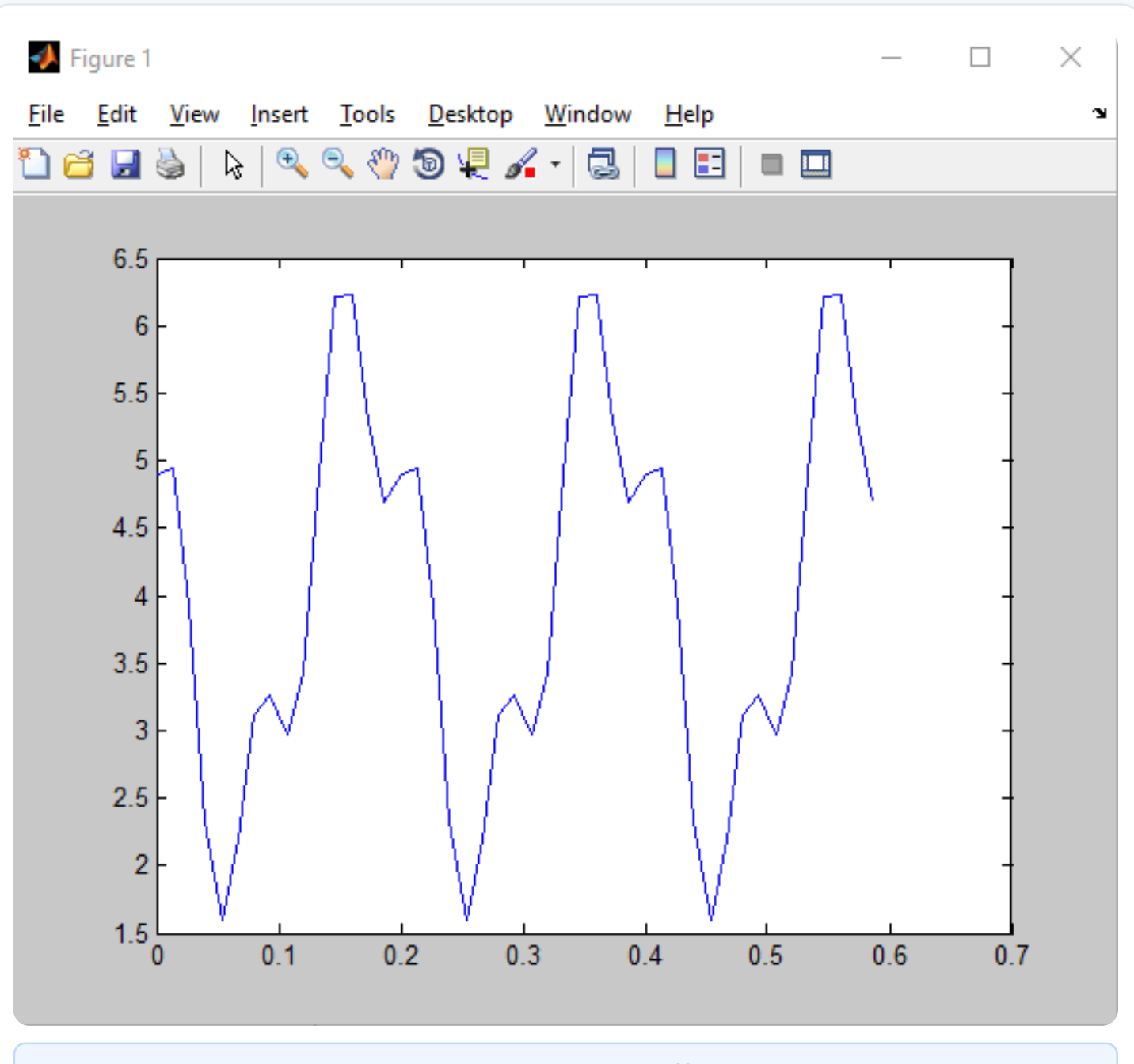
Le signal étudié :

$$x(t) = 4 + 1.8\cdot\cos(2\pi f_0 t + \pi/3) + 0.8\cdot\sin(6\pi f_0 t)$$

On a **f_0 = 5 Hz**, donc la période fondamentale $T_0 = 1/f_0 = 0.2$ s.
Ce signal est composé d'un *offset DC* (valeur 4), d'une composante à f_0 et d'une harmonique à $3f_0$.

▶ ÉTAPE 1 — TRAÇAGE DE X(T) AVEC DIFFÉRENTES VALEURS DE M

m contrôle le nombre de *périodes* tracées. Plus m est grand, plus on voit de répétitions du signal. **fe** est la fréquence d'échantillonnage : elle doit vérifier *fe ≥ 2·fmax* (théorème de Shannon) pour ne pas distordre le signal.

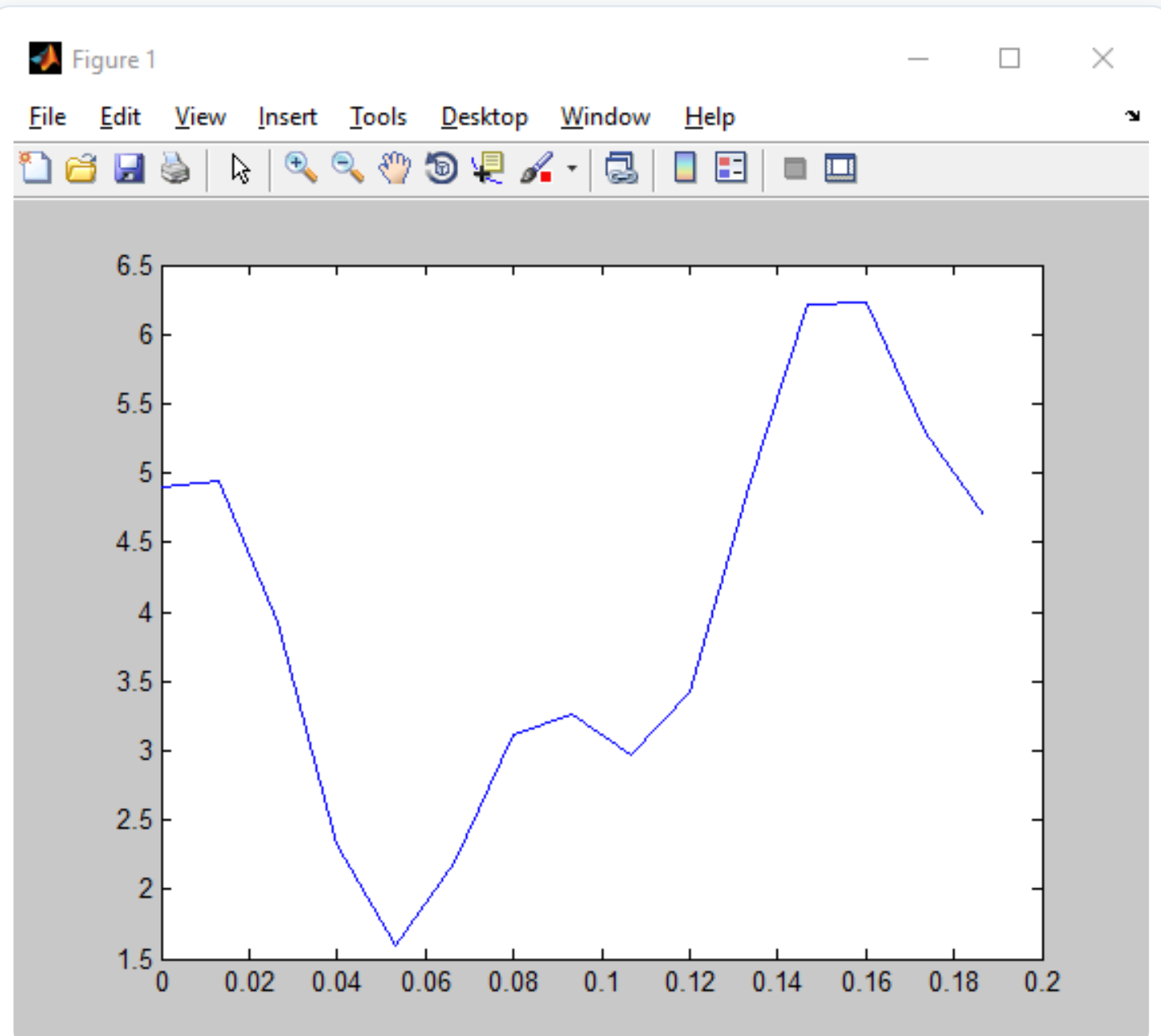


pour : m = 3 (3 périodes affichées)

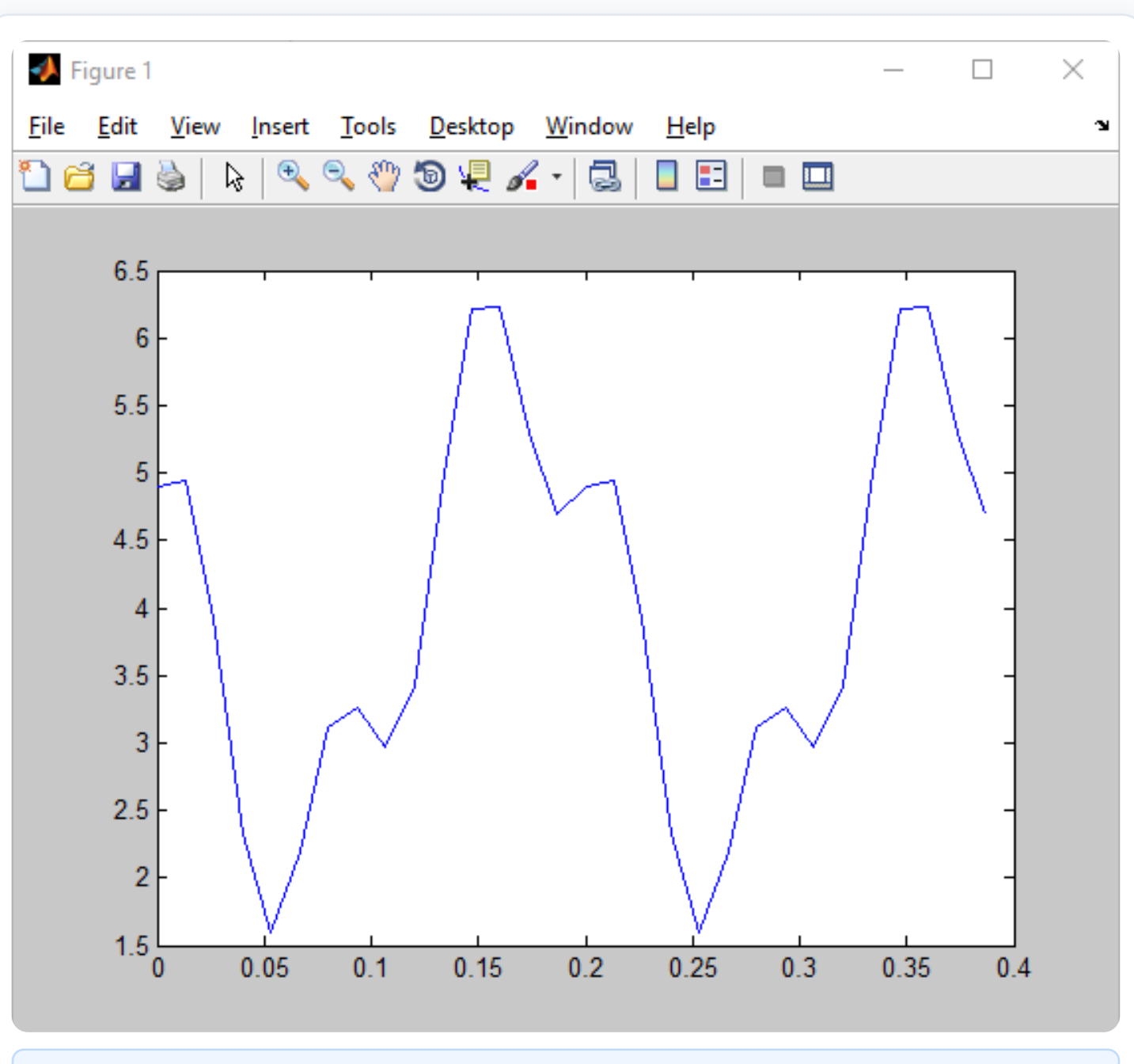
MATLAB — tp1_ex1.m

```
clear all
close all
clc

f0  = 5;          % fréquence fondamentale (Hz)
t0  = 1/f0;       % période fondamentale
m   = 3;           % ← nombre de périodes tracées
T   = m*t0;      % durée totale du signal
fe  = 15*f0;      % fréquence d'échantillonnage
te  = 1/fe;       % pas temporel (période d'échantillonnage)
t   = 0:te:T-te;  % axe du temps

x = 4 + 1.8*cos(2*pi*f0*t + pi/3) ...
       + 0.8*sin(6*pi*f0*t);

figure(1)
plot(t, x)
```

Le paramètre qui change : la valeur surlignée `m` contrôle combien de périodes on trace. Essayez m = 1, 2, 3 pour observer.



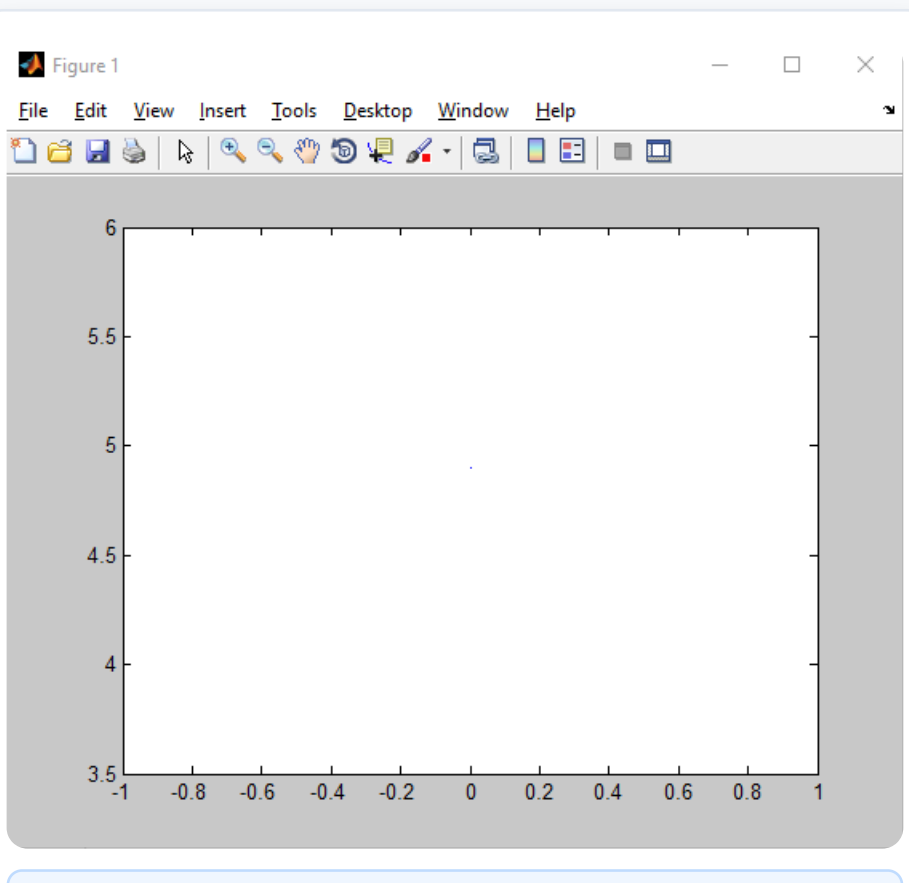
pour : m = 1



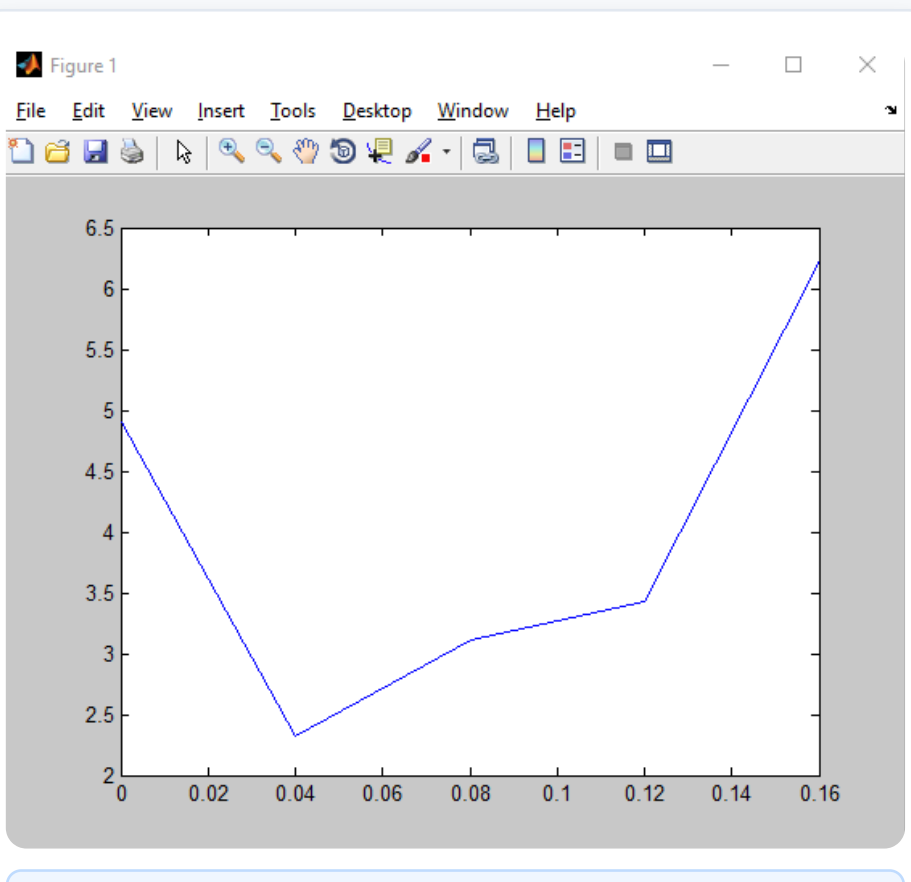
pour : m = 2

▶ EFFET DE LA FRÉQUENCE D'ÉCHANTILLONNAGE FE

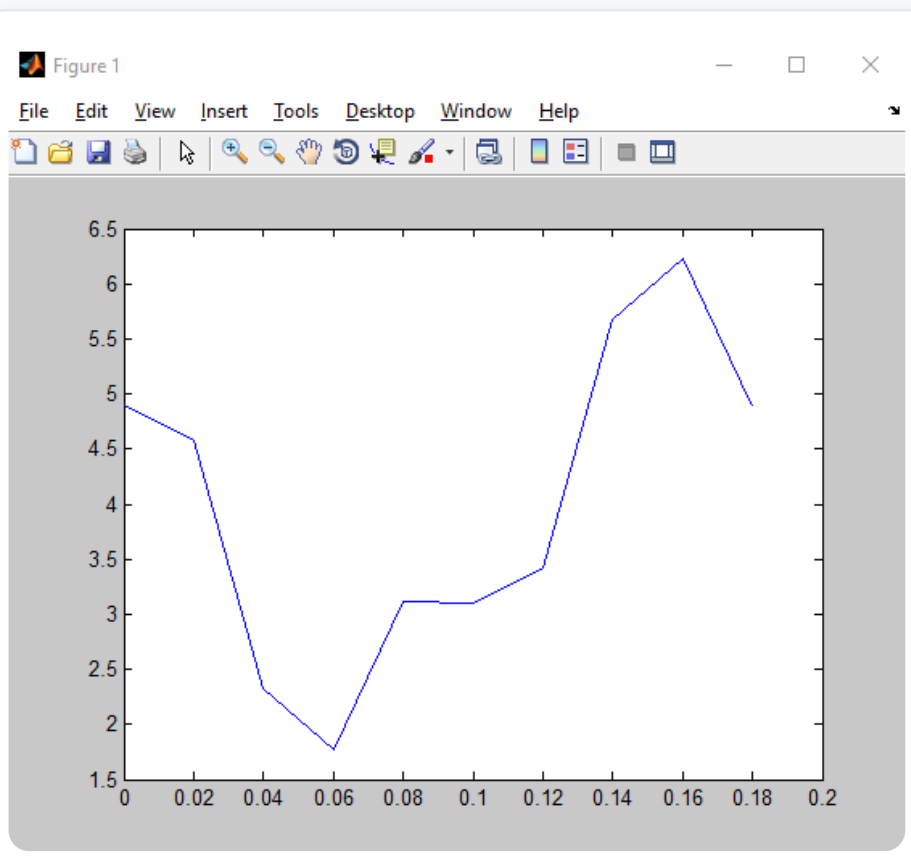
⚠ Si **fe = f0**, on sous-échantillonne → le signal est complètement déformé (aliasing) ! Il faut au minimum **fe ≥ 2·fmax = 6f0**. Ici on recommande **fe = 30·f0** pour un rendu propre.



Pour : fe = f0 ← ALIASING !



pour : fe = 5·f0



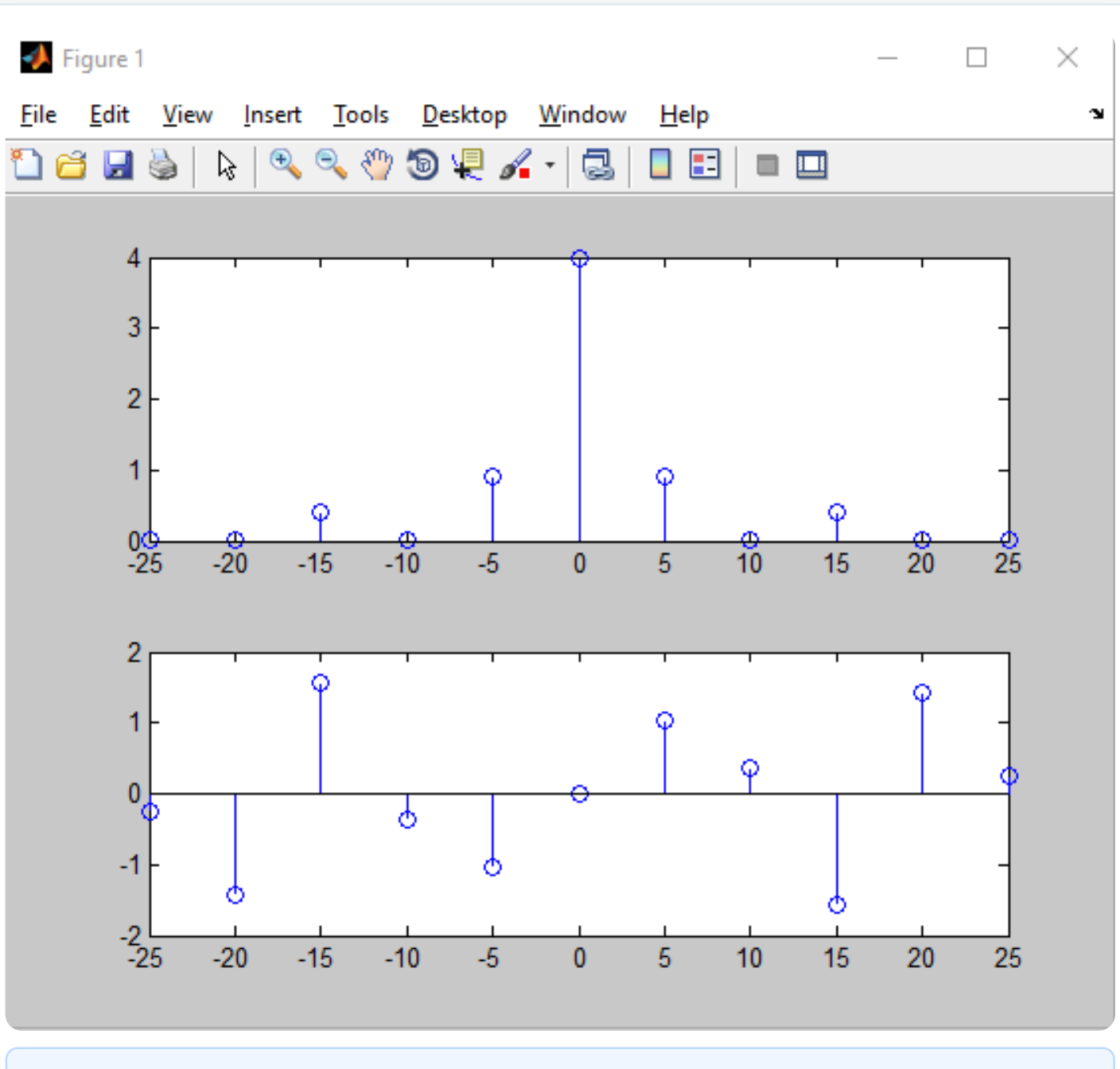
Pour : fe = 10·f0

▶ ÉTAPE 2 — CALCUL DES COEFFICIENTS DE FOURIER COMPLEXES X_K

On calcule numériquement l'intégrale de Fourier :

$$X_k = (t_e/t_0) \cdot \Sigma\, x[n] \cdot e^{(-j\cdot 2\pi k\cdot f_0\cdot t_n)}$$

On balaye k de −kmax à +kmax. Ici **kmax = 5** dans un premier temps.

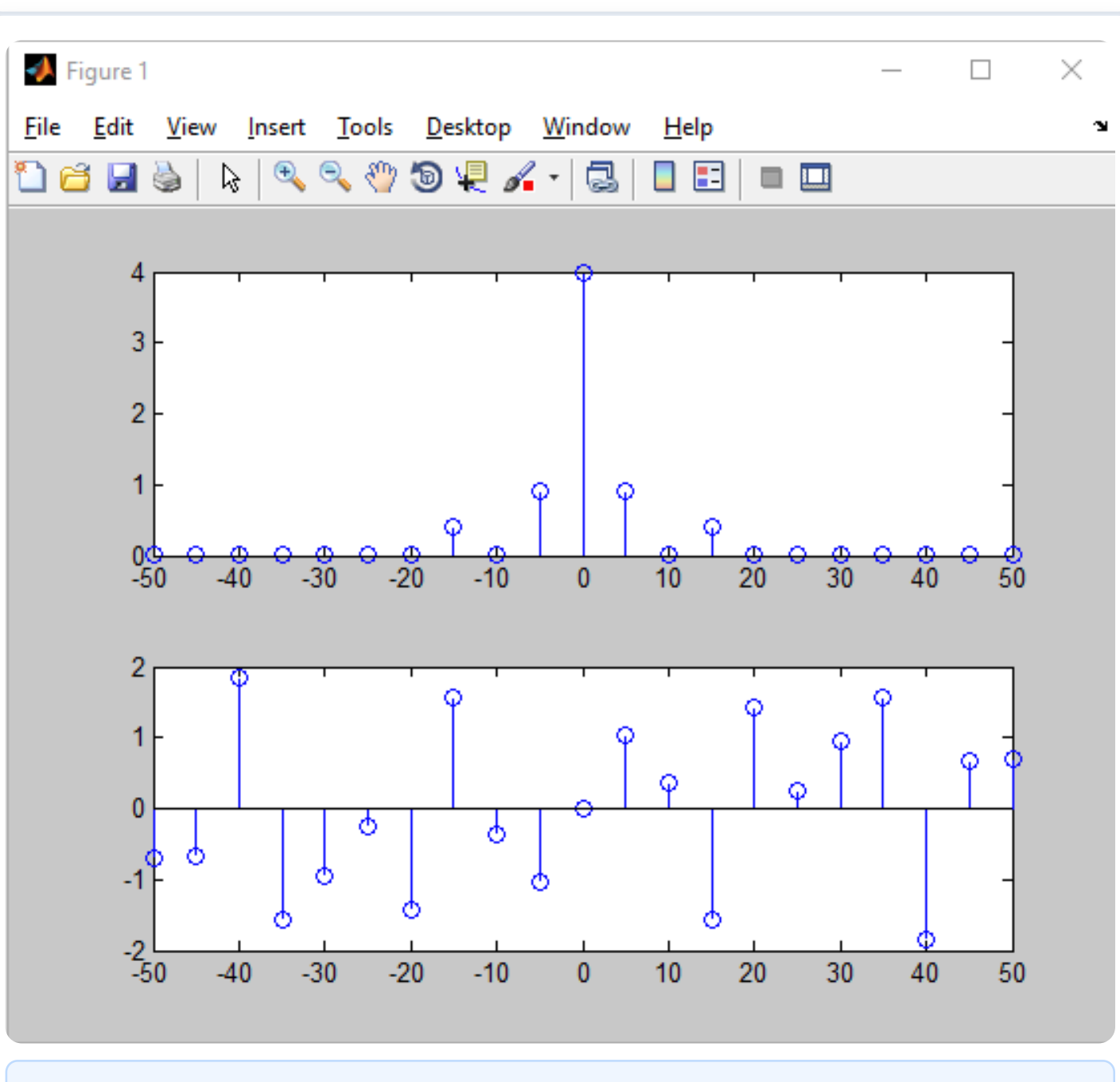


Spectres bilatéraux — kmax = 5

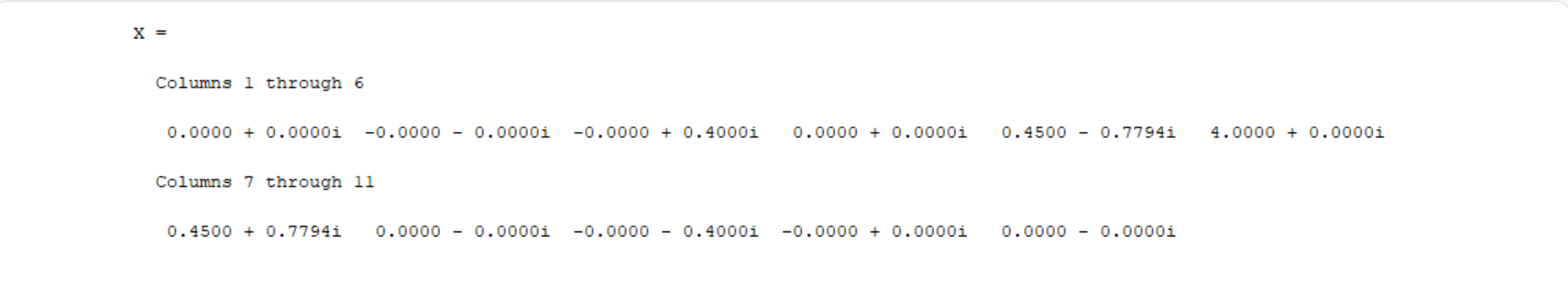
MATLAB — DSF Complexe

```
%% Calcul des coefficients Xk (DSF complexe)
kmax = 5;    % ← changer ici pour kmax=10

for k = -kmax : kmax
    k1 = k + kmax + 1;      % indice Matlab (≥1)
    X(k1) = (te/t0) * ...
        sum(x .* exp(-1j*2*pi*k*f0*t));
end

%% Spectres bilatéraux
absX   = abs(X);      % amplitude
alphaX = angle(X);    % phase
k = -kmax : kmax;
f = f0 * k;           % axe fréquentiel (Hz)

figure(2)
subplot(2,1,1); stem(f, absX);
subplot(2,1,2); stem(f, alphaX);
```

Résultats X (kmax=5) :
Les seuls coefficients non-nuls sont à k=−3, −1, 0, +1, +3 — ce qui correspond bien aux composantes du signal original (DC à k=0, cos à k=±1, sin à k=±3).



Spectres bilatéraux — kmax = 10

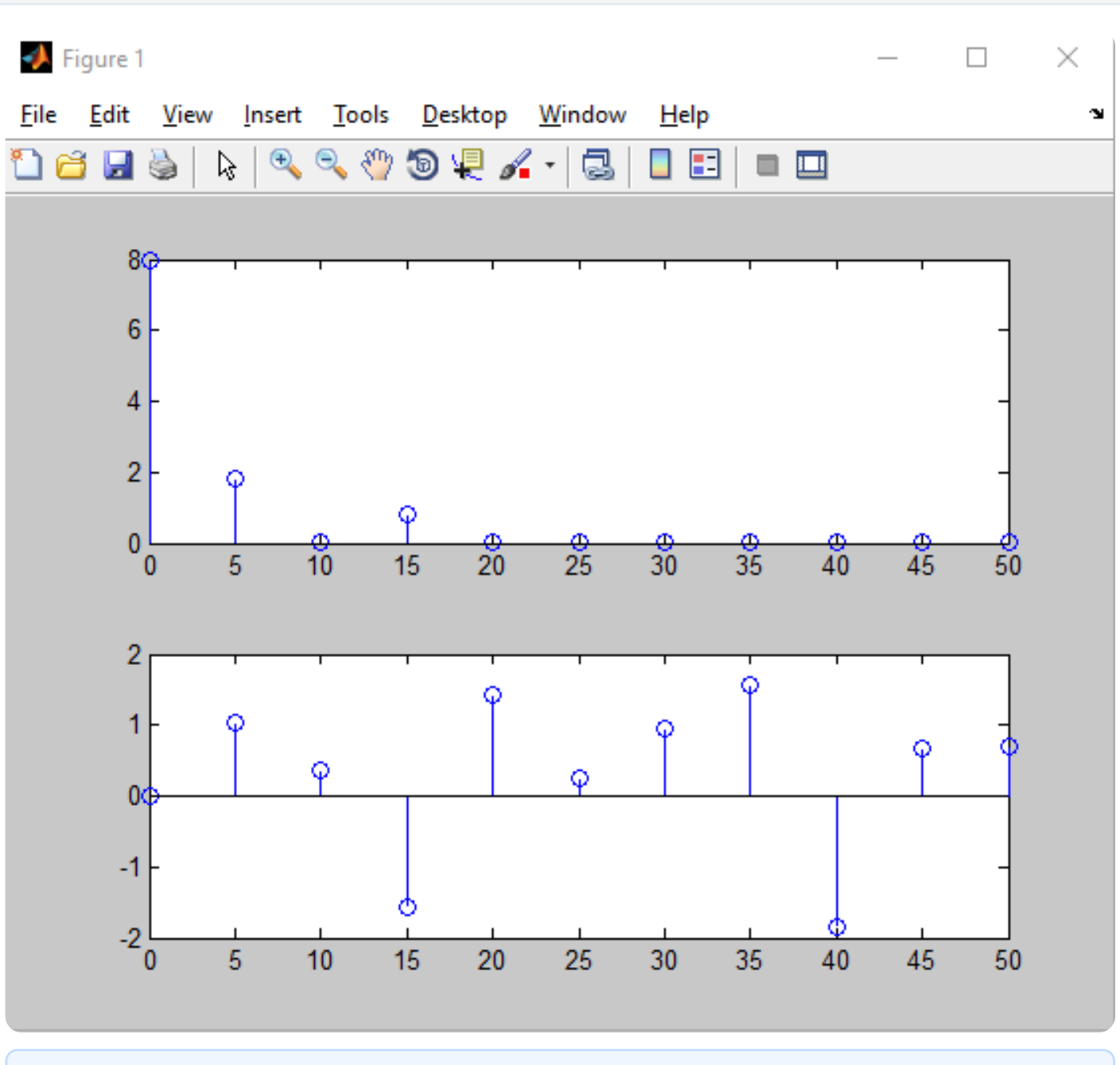


```
X =

  Columns 1 through 6

   0.0000 + 0.0000i  -0.0000 - 0.0000i  -0.0000 + 0.4000i   0.0000 + 0.0000i   0.4500 - 0.7794i   4.0000 + 0.0000i

  Columns 7 through 11

   0.4500 + 0.7794i   0.0000 - 0.0000i  -0.0000 - 0.4000i  -0.0000 + 0.0000i   0.0000 - 0.0000i
```

Sortie Matlab : Valeurs numériques du vecteur X (affichage console)

```
>> X
X =
  Colonnes 1-6  :   0+0i    0+0i    -0+0.4i    0+0i    0.45-0.78i    4+0i
  Colonnes 7-11 :   0.45+0.78i  0-0i    -0-0.4i    0+0i    0-0i
```

▶ ÉTAPE 3 — DSF DE BASE (A_K, B_K) ET DSF EN COSINUS (A_K, A_K)

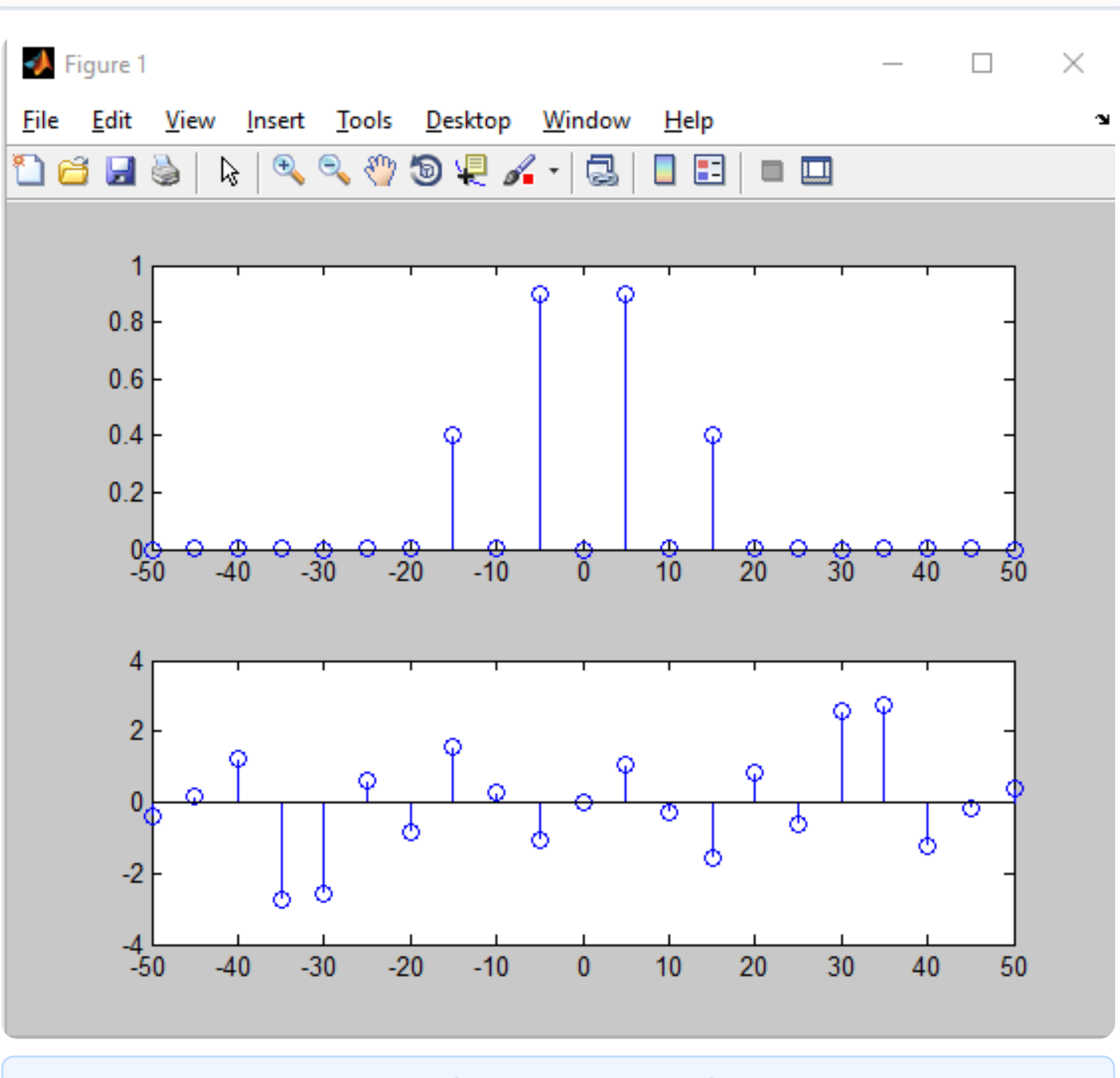


Spectres unilatéraux (k ≥ 0)

MATLAB — Autres DSF

```
%% Relations entre DSF (depuis les Xk)
% Garde seulement k ≥ 0 → partie "kmax+1 : end"
ak = 2*real(X(kmax+1:end)); % coeff cosinus
bk = -2*imag(X(kmax+1:end)); % coeff sinus
Ak = 2*absX(kmax+1:end);     % amplitude
ck = alphaX(kmax+1:end);       % phase

k = 0:kmax;
f = f0 * k;

figure(3)
subplot(2,1,1); stem(f, Ak);  % spectre amplitude
subplot(2,1,2); stem(f, ck);  % spectre phase

%% Exemple : supprimer la composante DC pour visualiser
x = 1.8*cos(2*pi*f0*t + pi/3) + 0.8*sin(6*pi*f0*t);
% ↑ on enlève le 4 pour mieux voir les oscillations
```

Relations clés :
- $a_0 = 2\cdot\mathrm{Re}(X_0)$ → composante continue (DC)
- $a_k = 2\cdot\mathrm{Re}(X_k)$ et $b_k = -2\cdot\mathrm{Im}(X_k)$
- $A_k = 2\cdot|X_k|$ (amplitude du cosinus)

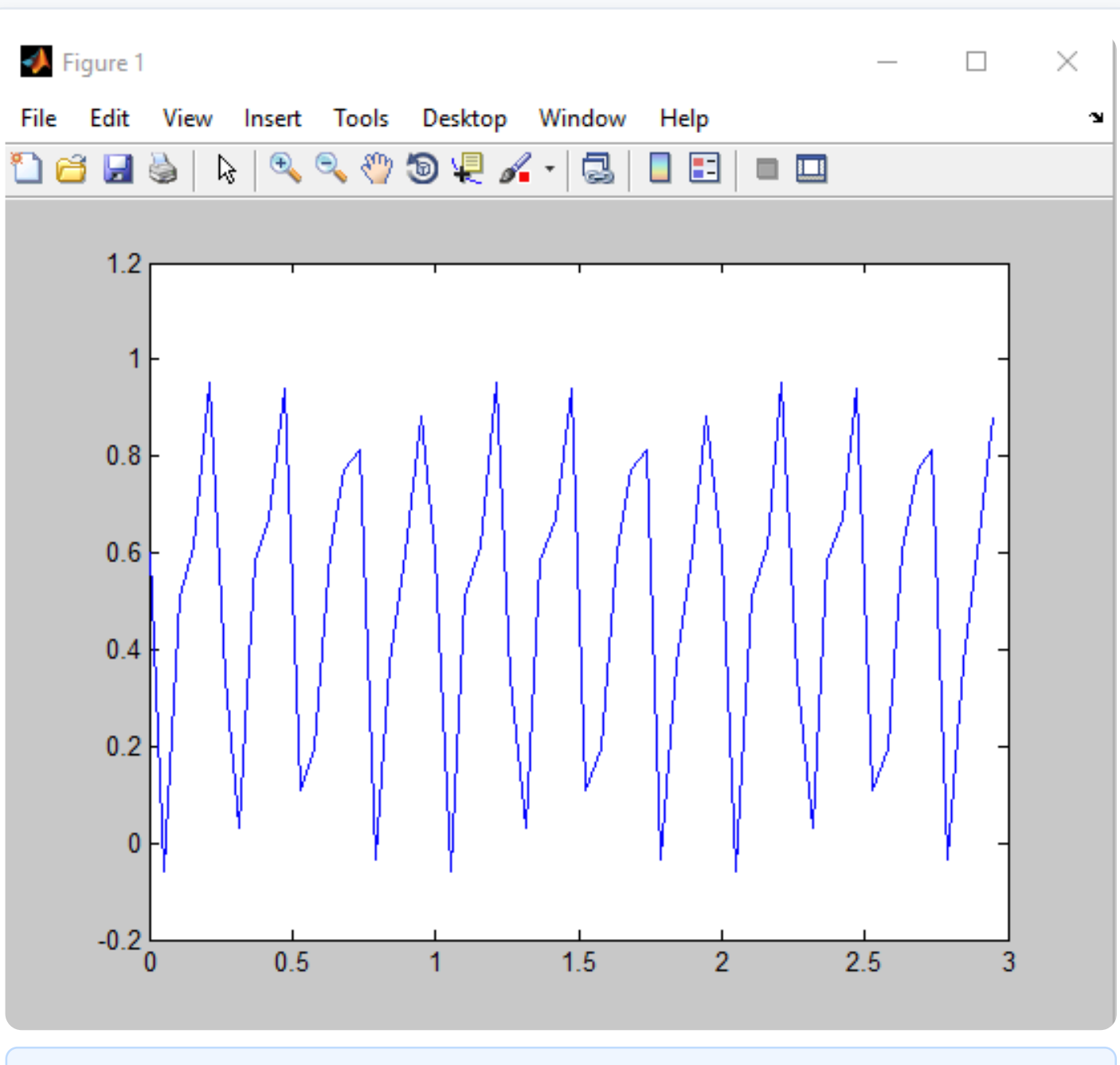


x sans DC : $1.8\cdot\cos(2\pi f_0 t+\pi/3) + 0.8\cdot\sin(6\pi f_0 t)$ — en élimine 4

Exercice 2 — Analyse d'un signal *inconnu*

On charge un signal dont on ne connaît *ni l'expression* ni la période. L'objectif est de retrouver son expression analytique grâce à l'analyse spectrale. Le fichier **x_inconnu.mat** contient le signal x et la fréquence `fe`.

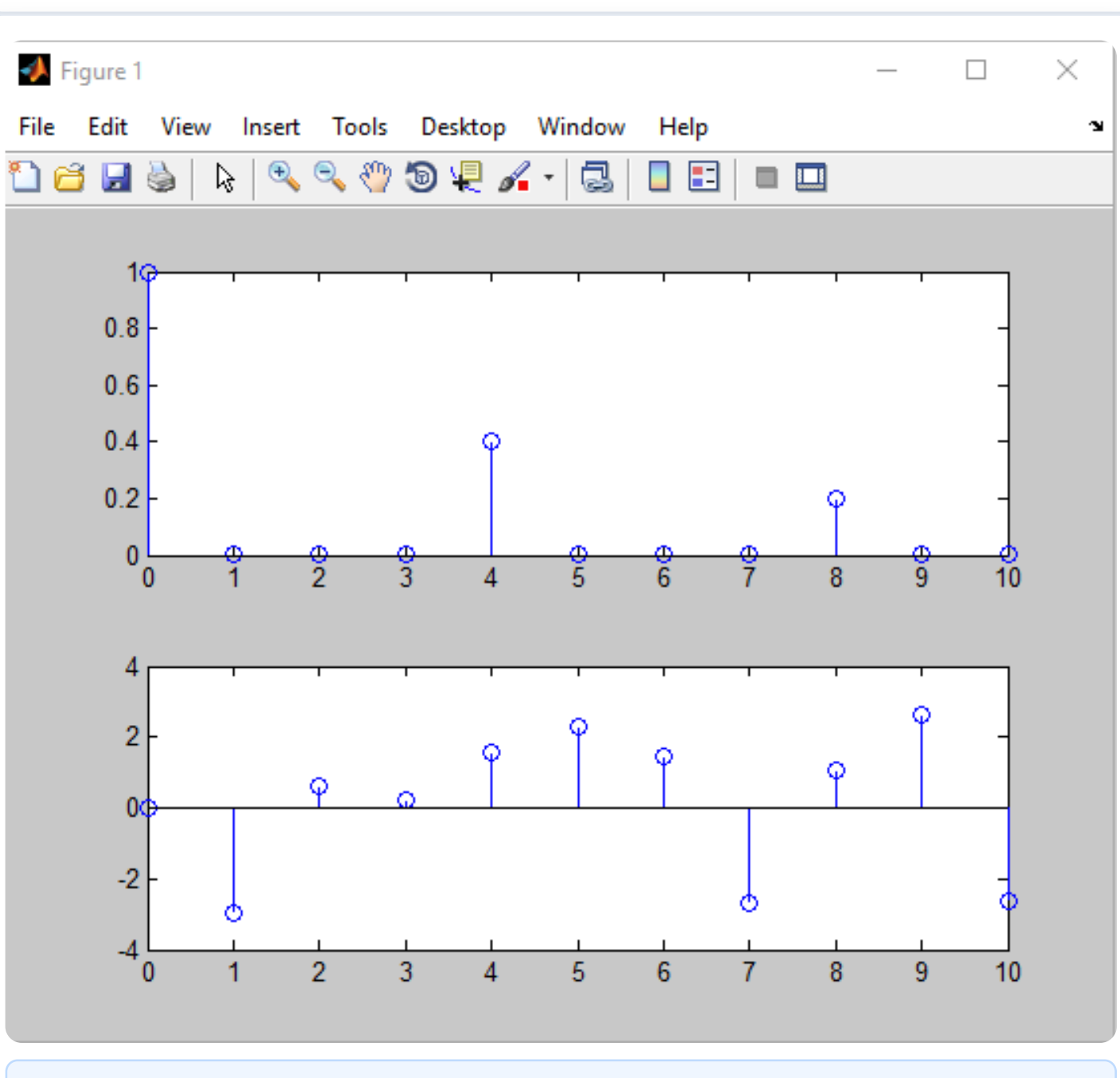
▶ CHARGEMENT ET VISUALISATION



Signal inconnu brut (durée totale)

MATLAB — tp1_ex2.m (complet)

```
clear all
close all
clc

%% 1. Chargement du signal inconnu
xinc = load('x_inconnu.mat');
x  = xinc.x;
fe = xinc.fe;
te = 1/fe;
N  = length(x);     % nombre total d'échantillons
T  = N*te;          % durée totale du signal
t  = 0:te:T-te;

figure(1); plot(t, x);  % visualiser pour trouver T0

%% 2. Identification de T0 et f0 (par observation)
t0 = 1;             % ← T0 lu graphiquement
f0 = 1/t0;

%% 3. Isoler une seule période
t = 0:te:t0-te;
x = x(1:length(t))'; % une seule période

%% 4. Calcul des coefficients Xk
kmax = 10;
for k = -kmax:kmax
    k1 = k + kmax + 1;
    X(k1) = (te/t0)*sum(x.*exp(-1j*2*pi*k*f0*t));
end

%% 5. Spectres bilatéraux
absX   = abs(X);
alphaX = angle(X);
k = -kmax:kmax;
f = f0*k;
subplot(2,1,1); stem(f, absX);
subplot(2,1,2); stem(f, alphaX);

%% 6. Spectres unilatéraux (ak, Ak ...)
ak = 2*real(X(kmax+1:end));
bk = -2*imag(X(kmax+1:end));
Ak = 2*absX(kmax+1:end);
ck = alphaX(kmax+1:end);
k  = 0:kmax;   f = f0*k;
subplot(2,1,1); stem(f, Ak);
subplot(2,1,2); stem(f, ck);
```



Signal inconnu — 1 seule période (T_0 = 1 s)

▶ RÉSULTATS NUMÉRIQUES ET EXPRESSION TROUVÉE

```
>> x (premiers échantillons)
x =
  0.6000  -0.0581   0.5093   0.6134   0.9533
  0.3427   0.0313   0.5850   0.6711   0.9381
  0.1110   0.1934   0.6042   0.7715   0.8152
 -0.0337   0.3701   0.6001   0.8823
```

```
>> ak (coefficients de la DSF de base)
ak =
  1.0000  0.0000  0.0000  0.0000  0.0000
  0.0000  0.0000  0.0000  0.1000  0.0000  0.0000
```

Lecture des coefficients :

Seuls deux coefficients non-nuls apparaissent :
- **a_0 = 1** → composante continue (DC)
- **a_8 = 0.1** → harmonique à $8f_0$

On peut alors écrire :

$$x(t) = 0.5 + 0.1\cdot\cos(2\pi\cdot 8\cdot f_0\cdot t)$$

(car a_0 correspond à $2\cdot X_0 = 2\times 0.5 = 1$)

L'analyse spectrale a permis de *retrouver l'expression analytique* du signal uniquement à partir de ses échantillons !

Code principal complet — Exercice 1 (avec éléments modifiables surlignés)

Les parties `surlignées en jaune` sont les lignes que vous devez **modifier** selon la question. Les parties `en vert` sont des ajouts optionnels.

MATLAB — tp1_ex1.m (code complet)

```
clear all; close all; clc

%% ── Paramètres ──────────────────────────────────
f0  = 5;           % fréquence fondamentale [Hz]
t0  = 1/f0;        % période fondamentale [s]
m   = 3;            % ← MODIFIER : nb de périodes (1, 2, 3...)
T   = m * t0;       % durée totale
fe  = 30*f0;        % ← MODIFIER : fe = f0 / 5*f0 / 10*f0 / 30*f0
te  = 1/fe;
t   = 0:te:T-te;

%% ── Signal ──────────────────────────────────────
x = 4 + 1.8*cos(2*pi*f0*t + pi/3) + 0.8*sin(6*pi*f0*t);
figure(1); plot(t, x); title('Signal x(t)')

%% ── DSF Complexe ────────────────────────────────
kmax = 5;            % ← MODIFIER : essayer 5, 10, 20...
for k = -kmax:kmax
    k1 = k + kmax + 1;
    X(k1) = (te/t0) * sum(x .* exp(-1j*2*pi*k*f0*t));
end

%% ── Spectres bilatéraux ─────────────────────────
absX   = abs(X);
alphaX = angle(X);

% Seuillage pour nettoyer la phase numérique
eps_seuil = 1e-5;
ind = find(absX < eps_seuil);
alphaX(ind) = 0;

k = -kmax:kmax;
f = f0 * k;

figure(2)
subplot(2,1,1); stem(f, absX);  title('|Xk| bilatéral')
subplot(2,1,2); stem(f, alphaX); title('Phase bilatérale')

%% ── DSF base (ak, bk) et cosinus (Ak, ak) ───────────
ak = 2*real(X(kmax+1:end));
bk = -2*imag(X(kmax+1:end));
Ak = 2*absX(kmax+1:end);
ck = alphaX(kmax+1:end);

k = 0:kmax;
f = f0 * k;

figure(3)
subplot(2,1,1); stem(f, Ak);  title('|Ak| unilatéral')
subplot(2,1,2); stem(f, ck); title('Phase unilatérale')

%% ── Puissance (Parseval) ────────────────────────
Pt = mean(x.^2);              % puissance temporelle
Pf = sum(abs(X).^2);          % puissance spectrale
fprintf('Pt = %.4f  |  Pf = %.4f
', Pt, Pf);
```

Mémo théorique — Formules essentielles

DSF Complexe

$$X_k = (1/T_0) \int x(t)\cdot e^{(-j2\pi k f_0 t)}\, dt$$

$$x(t) = \Sigma\, X_k \cdot e^{(+j2\pi k f_0 t)}$$

$k \in \mathbb{Z}$

DSF en cosinus

$$x(t) = A_0 + \Sigma\, A_k\cdot\cos(2\pi k f_0 t + \alpha_k)$$

$$A_k = 2|X_k|\ ;\ \alpha_k = \arg(X_k)$$

$k \geq 1$

DSF de base (sinus-cosinus)

$$x(t) = a_0/2 + \Sigma\, [a_k\cos(2\pi k f_0 t) + b_k\sin(2\pi k f_0 t)]$$

$$a_k = 2\mathrm{Re}(X_k)\ ;\ b_k = -2\mathrm{Im}(X_k)$$

Théorème de Parseval

$$P = (1/T_0)\int|x(t)|^2\, dt = \Sigma\, |X_k|^2$$

La puissance totale peut se calculer dans le domaine temporel *ou* fréquentiel. Si les deux valeurs sont égales → Parseval vérifié ✓

▶ RAPPEL : COMMANDES MATLAB CLÉS

`abs(X)` → module de X
`real(X)` → partie réelle
`stem(f, Y)` → spectre discret
`sum(x .* exp(...))` → intégrale discrète
`angle(X)` → phase de X (en rad)
`imag(X)` → partie imaginaire
`subplot(m,n,p)` → multi-figures
`find(absX < eps)` → seuillage phase